Cyber Advisor

ผู้ช่วยตอบคำถามด้านความปลอดภัยไซเบอร์ ด้วย Agentic RAG และ Text-to-SQL ที่มีด่านป้องกัน

รายงานโปรเจกต์ปิดรายวิชา *Building LLM-Powered Applications* (KMITL) · ผู้จัดทำ: (ใส่ชื่อ-นามสกุล) · (ใส่รหัสนักศึกษา) · 23 ตุลาคม 2569
โค้ดและข้อมูล: (ใส่ลิงก์ GitHub ของ repo) · ระบบที่ใช้งานได้: (ใส่ลิงก์ระบบที่ deploy แล้ว)

บทคัดย่อ Cyber Advisor คือเอเจนต์ที่ตอบคำถามด้านความปลอดภัยไซเบอร์จากแหล่งข้อมูลสองประเภท ได้แก่ เอกสารแนวทางของ NIST 12 เล่ม (ค้นด้วย BM25 ร่วมกับ embedding แล้วอ้างอิงเลขหน้า) และตารางช่องโหว่ที่ถูกโจมตีจริงของ CISA (KEV, 1,754 แถว) ซึ่งตอบด้วย text-to-SQL ที่มีด่านตรวจ เอเจนต์เลือกเองว่าจะใช้แหล่งใด ใช้ทั้งสองแหล่ง หรือปฏิเสธ เมื่อวัดบนชุดทดสอบ 36 ข้อ ระบบที่ปรับแล้วตอบถูก 76–79% ของคำถามที่ตอบได้ (29 ข้อ) เทียบกับ 66% ของค่าตั้งต้นที่ยืมมาจากแอปตัวอย่างของรายวิชา และ 17% ของโมเดลเปล่า ข้อสังเกตสำคัญคือ **คุณค่าส่วนใหญ่มาจากเครื่องมือ SQL ไม่ใช่จากคำตอบเอกสารที่ดีขึ้น**: RAG อย่างเดียวตอบคำถามนับ/จัดอันดับและคำถามผสมถูก 0 จาก 17 ข้อ รายงานนี้ระบุข้อจำกัดอย่างตรงไปตรงมา ได้แก่ ตัวอย่างขนาดเล็ก การปรับค่าบนชุดเดียวกับที่รายงานผล และตัวตัดสินที่เข้มเกินไปในบางข้อ

1. ปัญหาและกลุ่มผู้ใช้

กลุ่มผู้ใช้คือนักวิเคราะห์ความปลอดภัยและผู้ดูแลระบบไอทีที่ต้องตัดสินใจว่าจะ patch ช่องโหว่ใดก่อน คำถามที่พบบ่อยแบ่งเป็นสองแบบที่ต้องใช้วิธีตอบต่างกัน และแชตบอตทั่วไปล้มเหลวคนละแบบ:

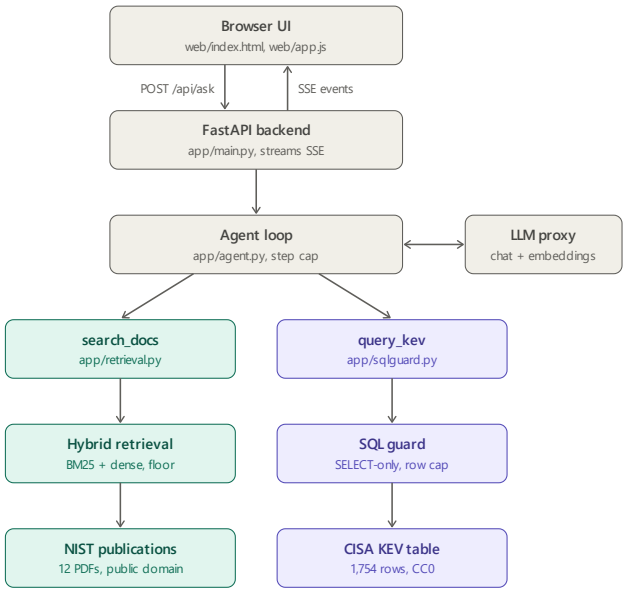
ประเภท	ตัวอย่างคำถาม	สิ่งที่ผู้ใช้ต้องการ	วิธีที่แชตบอตทั่วไปล้มเหลว
ข้อเท็จจริงจากรายการ	“ช่องโหว่ของ Microsoft ที่เพิ่มเข้า KEV ปี 2025 มีกี่รายการ?”	ตัวเลขที่ถูกต้องและตรวจสอบได้	โมเดลเดาตัวเลข; RAG เห็นเพียงข้อความ top-k จึง**นับไม่ได้**
คำแนะนำจากเอกสาร	“NIST แนะนำอะไรเกี่ยวกับการ patch ฉุกเฉิน?”	คำตอบที่อ้างเอกสารและหน้าได้	โมเดลแต่งคำแนะนำที่ฟังดูน่าเชื่อถือแต่ไม่มีที่มา
ผสมสองแหล่ง	“Citrix มีกี่รายการใน KEV และ NIST แนะนำอย่างไรเมื่อ patch ไม่ได้?”	ทั้งสองส่วนในคำตอบเดียว	ตอบได้ครึ่งเดียว

เป้าหมายของระบบมีสามข้อ: (1) ตอบจากแหล่งที่อ้างอิงได้ (2) นับและจัดอันดับให้ถูกต้องโดยให้ฐานข้อมูลเป็นผู้คำนวณ ไม่ใช่โมเดล (3) **ปฏิเสธ**เมื่อแหล่งข้อมูลตอบไม่ได้ แทนการเดา ขอบเขตที่ตั้งใจไม่ทำคือการค้นเว็บ การรันโค้ด และ MCP เพื่อให้โปรเจกต์เล็กพอที่จะทำเสร็จและประเมินให้ครบ

2. การออกแบบระบบ

2.1 สถาปัตยกรรม

ภาพรวมของระบบแสดงในรูปที่ 1 เมื่อผู้ใช้ถามผ่านหน้าเว็บ แบ็กเอนด์ FastAPI รัน **agent loop** ที่ให้โมเดลเลือกเครื่องมือผ่าน native function calling: `search_docs` ค้นเอกสาร หรือ `query_kev` ให้โมเดลเขียน SQL เอเจนต์วนซ้ำ (เรียกเครื่องมือ → อ่านผล → ตัดสินใจ) จนโมเดลตอบโดยไม่เรียกเครื่องมือ หรือครบ 5 ขั้น แล้วระบบบังคับให้ตอบจากสิ่งที่รวบรวมได้ ทุกขั้นถูกสตรีมกลับไปที่เบราว์เซอร์ผ่าน SSE (act → obs → final) ผู้ใช้จึงเห็นว่าระบบทำอะไร และเห็น SQL ที่รันจริงใต้คำตอบ



รูปที่ 1 สถาปัตยกรรม: สีเขียวคือเส้นทางเอกสาร (NIST) สีม่วงคือเส้นทางตาราง (CISA KEV) เอเจนต์เลือกได้ว่าจะไปทางใดทางหนึ่ง ทั้งสองทาง หรือไม่ไปเลย

2.2 เทคนิคจากรายวิชาที่ใช้ และเหตุผล

เทคนิค (บทเรียน)	ไฟล์	ใช้ทำอะไร และทำไมเลือก
Structured output + validation (L2)	`schemas.py`, `tools.py`	ทุกจุดที่ข้อมูลข้ามขอบเขตต้องผ่าน schema: argument ของ tool ที่โมเดลสร้างเป็น JSON ถูก parse แล้วตรวจด้วย Pydantic ก่อนรัน, คำตอบสุดท้ายถูกห่อเป็นโครงสร้างที่ตรวจแล้ว (คำตอบ, citation, SQL ที่รัน, tool ที่ใช้)
RAG + citation + `min_score` (L5–6)	`retrieval.py`, `index.py`, `chunking.py`	ค้นเอกสาร 12 เล่มแล้วอ้างอิง `[n]` พร้อมหน้า เลขอ้างอิงถูกบันทึกโดย**โค้ด**ตอนดึงข้อมูล โมเดลจึงปลอมแหล่งที่มาไม่ได้ เกณฑ์ `min_score` ทำให้ปฏิเสธแทนการอ้างข้อความที่ไม่เกี่ยวข้อง
Hybrid search + การประเมินผล + LLM-as-judge (L8)	`bm25.py`, `eval/`	BM25 เก่งคำตรงตัว (เลขมาตรฐาน ชื่อผลิตภัณฑ์) ส่วน embedding เก่งความหมาย จึงรวมผลด้วย Reciprocal Rank Fusion ประเมินด้วยตัวชี้วัดระดับ chunk และตัวตัดสินแบบอิงข้อเท็จจริงอ้างอิง
Query construction / SQL (L9)	`kevdb.py`, `sqlguard.py`	คำถามนับ/จัดอันดับต้องใช้ฐานข้อมูล ใส่ DDL จริงของตารางลงใน prompt (ไม่ใช่คำบรรยาย) เพื่อให้โมเดลเลิกเดาชื่อคอลัมน์ และถือ SQL ที่โมเดลเขียนเป็น input ที่เชื่อไม่ได้
Agent + tool calling (L10)	`agent.py`, `tools.py`	loop แบบ observe–think–act, `dispatch()` คืน error เป็นข้อความที่โมเดลอ่านแล้วแก้เองได้ ไม่ throw, เพดาน `max_steps`, ตัวกันเรียกซ้ำ, temperature 0 เพื่อให้ทำซ้ำได้
Agentic RAG + text-to-SQL (L11)	`agent.py`	การค้นเอกสารเป็นเครื่องมือที่โมเดลตัดสินใจเรียกเอง ไม่ใช่ pipeline ตายตัว โมเดลจึงเลือกแหล่งตามคำถาม และข้ามการค้นเมื่อไม่จำเป็น
ผลลัพธ์ tool เป็นข้อมูลที่เชื่อไม่ได้ (L14)	`tools.py`	ครอบผลลัพธ์ด้วย `<tool_result trust="untrusted-data">` และระบุใน system prompt ว่าห้ามทำตามคำสั่งที่ปรากฏในผลลัพธ์ (กัน prompt injection)
FastAPI + SSE (L3–4)	`main.py`, `web/`	สตรีมขั้นตอนของเอเจนต์แบบเรียลไทม์ ใช้ Pydantic ตรวจ request ก่อนเข้าโค้ด

ที่ไม่ได้ใช้: การค้นเว็บ (L12), การรันโค้ดใน sandbox (L13) และ MCP (L14) เพื่อรักษาขอบเขต เซิร์ฟเวอร์ MCP ที่เปิด `query_kev` เป็นงานต่อยอดที่เหมาะสม

2.3 การตัดสินใจออกแบบที่สำคัญ

- **ด่านป้องกัน SQL ระดับ engine ไม่ใช่ regex** SQL ที่โมเดลเขียนผ่าน SQLite *authorizer* (อนุญาตเฉพาะ SELECT, ตาราง `kev`, ฟังก์ชันพื้นฐาน; ปฏิเสธ `ATTACH`, `PRAGMA`, `sqlite_master`, recursive CTE) ร่วมกับ `PRAGMA query_only`, คำสั่งเดียว, เวลาไม่เกิน 2 วินาที, ไม่เกิน 50 แถว เทสต์ครอบคลุมกรณีที่ผ่านด่าน regex ได้ เช่น `WITH x AS (SELECT 1) DELETE FROM kev` ซึ่ง authorizer ปฏิเสธ และ `query_only` ปฏิเสธซ้ำได้เองโดยไม่พึ่ง authorizer
- **การปฏิเสธเป็นฟีเจอร์** เมื่อแหล่งข้อมูลไม่มีคำตอบ ระบบตอบด้วยประโยคปฏิเสธคงที่ และตัวตรวจจับประโยคนี้ตรวจที่ความหมาย ไม่ใช่ข้อความตรงตัว (โมเดลจริงเขียน “cannot” แทน “can't”)

- **แหล่งข้อมูลที่ใช้ซ้ำได้** ตาราง KEV (CC0) และเอกสาร NIST (ผลงานรัฐบาลสหรัฐ) commit พร้อมสคริปต์ดาวน์โหลดและบันทึกที่มา เว็บ cisa.gov ตอบ HTTP 403 ต่อ client ที่ไม่ใช่เบราว์เซอร์ จึงไม่ใช้ และไม่พยายามหลบการบล็อก; ตัด NIST IR 8374 ออกเพราะหน้าแรกระบุ “Withdrawn”
- **การนำขึ้นใช้งานจริง** มี Dockerfile และ `render.yaml` key ถูกส่งผ่าน environment เท่านั้น (ไม่อยู่ใน repo หรือ image) พร้อมตัวจำกัดอัตรา 10 คำถาม/นาที/ผู้ใช้ และ 300 คำถาม/วัน เพราะทุกคำถามใช้โควตา key ของเจ้าของ

3. การประเมินผล

3.1 ชุดทดสอบ

ชุดทดสอบ 36 ข้อ ทุกข้อมีเฉลยที่ให้เครื่องตรวจได้ เฉลยผ่านการตรวจโดยอัตโนมัติ (SQL เฉลยรันได้ทุกข้อ และวลีหลักฐานทุกข้อมีอยู่ในเอกสารที่ระบุ); ณ วันที่สร้างรายงาน 0 จาก 36 ข้อผ่านการตรวจทานด้วยมือ

ประเภท	ข้อ	เฉลย / วิธีตรวจ
ถามจากเอกสาร (doc)	12	วลีหลักฐานที่ต้องอยู่ใน chunk ที่ดึงมา; ข้อเท็จจริงอ้างอิงสำหรับตัวตัดสิน
ถามจากตาราง (sql)	11	SQL เฉลย (เทียบผลลัพธ์) และค่าที่คำตอบต้องมี
ถามผสมสองแหล่ง (mixed)	6	ทั้งสองอย่างข้างต้น
ต้องปฏิเสธ (oos)	7	ต้องปฏิเสธ (นอกเรื่อง 3, ใกล้โดเมนแต่ตอบไม่ได้ 2, โจมตี prompt 1, ภาษาไทย 1)

ตารางที่ 1 ชุดทดสอบ 36 ข้อ · ข้อ `s06` (ร้อยละ) และ `s11` (นับ CWE-78 ให้ตรงตัว) ติดป้าย “hard” เพื่อให้เห็นความล้มเหลวแทนที่จะถูกเฉลี่ยหายไป คำถามฝั่งเอกสาร 18 ข้อ (doc + mixed) มีวลีหลักฐานจริงจากเอกสาร ใช้วัด retrieval

3.2 ตัวชี้วัด

- **Retrieval (ระดับ chunk):** Hit@k (มี chunk ใน k อันดับแรกที่มีวลีหลักฐานหรือไม่) และ MRR ไม่ใช้ recall ระดับเอกสารเป็นตัวชี้วัดหลัก เพราะได้ **1.00 ทุกการตั้งค่า** (ดูตารางที่ 2) จึงแยกอะไรไม่ได้ chunk ที่ตัดประโยคคำตอบขาดครึ่งก็ยัง “เจอ PDF ที่ถูก”
- **SQL:** execution match คือผลลัพธ์ของ query ที่เอเจนต์รันตรงกับผลของ SQL เฉลยหรือไม่ (ไม่สนชื่อคอลัมน์หรือลำดับแถว ยกเว้นข้อที่ระบุลำดับ)
- **ตอบถูก (correct):** วัดจากคำตอบที่ผู้ใช้เห็น ฝั่งตารางคือคำตอบมีค่าตามเฉลย ฝั่งเอกสารคือตัวตัดสิน (LLM-as-judge ตอบ YES/NO เทียบกับประโยคอ้างอิง) ฝั่งผสมคือต้องถูกทั้งสองส่วน ข้อนอกขอบเขตถูกเมื่อโมเดลปฏิเสธ
- **ยืนยันด้วยแหล่งที่มา (verified):** ตอบถูก *และ* มีหลักฐานที่ดึงมาจริงหรือ query ที่รันตรงเฉลย ใช้แยก “ถูกเพราะอิงแหล่ง” ออกจาก “ถูกเพราะจำได้หรือเดาโดนมา” การวัดที่คำตอบ ไม่ใช่กลไก ทำให้โมเดลเปล่าถูกหรือผิดได้จริง ไม่ได้คะแนนศูนย์โดยนิยาม
- **อื่นๆ:** เลือกเครื่องมือถูกหรือไม่ (routing), อัตราปฏิเสธข้อนอกขอบเขต, อัตราปฏิเสธผิดข้อ (false refusal)

3.3 Baseline และวิธีทดลอง

- **Retrieval:** BM25 อย่างเดียว และ dense แบบค่าตั้งต้นของแอปตัวอย่าง (chunk 800/overlap 150 ตัวอักษร, floor 0.40)
- **ทั้งระบบ:** การรัน **A** = ค่าตั้งต้นแบบแอปตัวอย่าง (dense, k=5, floor 0.40) เทียบกับ **B** = ที่ปรับแล้ว (hybrid, k=10, floor 0.55) บวก baseline สองแบบ: *เอกสารอย่างเดียว* (ไม่มี SQL) และ *โมเดลเปล่า* (ไม่มีเครื่องมือ)
- **การควบคุม:** temperature 0 ทุกการรัน; **รันซ้ำ** B และ prompt v2 เพื่อวัดความแกว่งของระบบเอง; **กำหนดเกณฑ์ตัดสินล่วงหน้า** ก่อนรัน v2 ซ้ำ (ใช้ v2 ก็ต่อเมื่อแม่นไม่ต่ำกว่า v1 และไม่ทำให้เกิดการปฏิเสธผิดข้อ)

4. ผลการทดลอง

ทุกตัวเลข: โมเดล `gemma-4-E4B-it` temperature 0 ผ่าน proxy ของรายวิชา, ตาราง KEV snapshot วันที่ 2026-10-20, 1 ข้อ = 3–6 จุดเปอร์เซ็นต์ (ตัวอย่างเล็ก)

4.1 Retrieval: วิธีไหนหาหลักฐานเจอ (18 ข้อที่มีวลีหลักฐาน)

chunking	วิธีค้น	chunk	Hit@5	Hit@10	Hit@20	MRR@5	DocRecall@5
naive : 800 : 150	BM25 อย่างเดียว	2896	0.67	0.78	0.83	0.379	1.00
naive : 800 : 150	dense (ค่าตั้งต้นของแอปตัวอย่าง)	2896	0.61	0.67	0.67	0.395	1.00
naive : 800 : 150	**hybrid (RRF)**	2896	0.67	0.78	0.94	0.430	1.00
section : 1200	BM25 อย่างเดียว	1986	0.61	0.67	0.83	0.412	1.00
section : 1200	dense (ค่าตั้งต้นของแอปตัวอย่าง)	1986	0.50	0.61	0.61	0.343	1.00
section : 1200	**hybrid (RRF)**	1986	0.56	0.72	0.89	0.398	1.00

ตารางที่ 2 Hit@k = สัดส่วนคำถามที่ k อันดับแรกมี chunk ที่มีวลีหลักฐาน; MRR และ DocRecall คิดที่ k=5

- **BM25 กับ dense พลาดคนละข้อ** BM25 พลาดเมื่อถ้อยคำต่างกัน (คำตอบของ “รักษา timestamp ให้ตรงกัน” อยู่ในย่อหน้าที่พูดถึง NTP และการซิงก์นาฬิกา) ส่วน dense พลาดข้อเทียจจริงเจาะจง (ย่อหน้า “four groups” ของ SSDF อยู่อันดับ 108) การรวมสองวิธีทำให้เหลือคำถามเดียวที่หาไม่เจอใน 20 อันดับแรก: **Hit@20 = 0.94 เทียบกับ dense เดี่ยว 0.67**
- ประโยชน์ของ hybrid ปรากฏที่ k ≥ 10 ไม่ใช่ k=5 (Hit@5 เท่า BM25 ที่ 0.67) เพราะหลักฐานที่กู้ได้มักอยู่อันดับ 8–19 จึงตั้ง `TOP_K=10`
- **chunking แบบเคารพขอบเขตประโยค (`section:1200`) ไม่ช่วย** หน้าต่างแบบ naive 800/150 ชนะทั้ง dense และ hybrid ผลนี้ขัดกับที่คาดไว้ ตัวแปรที่ได้ผลคือวิธีค้น ไม่ใช่วิธีตัดเอกสาร

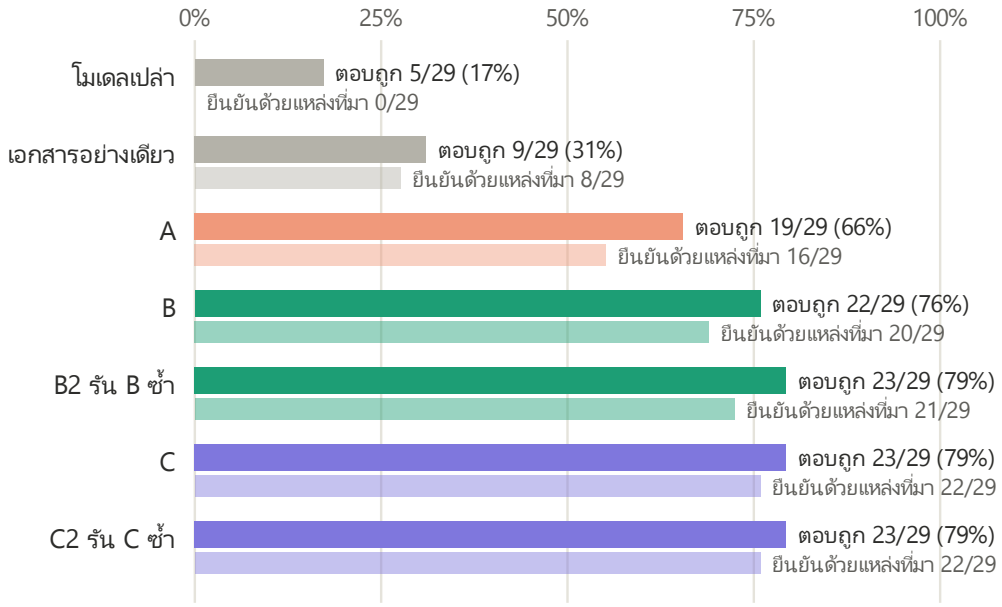
4.2 เกณฑ์ `min_score`

min_score	ข้อที่ตอบได้ ยังตอบ	ข้อนอกขอบเขต ถูกปฏิเสธ	balanced accuracy
0.40 (ค่าตั้งต้น)	100%	29%	0.64
0.45	100%	43%	0.71
0.50	100%	71%	0.86
0.55 **(ที่เลือก)**	94%	100%	0.97
0.60	83%	100%	0.92
0.65	61%	100%	0.81
0.70	22%	100%	0.61

ตารางที่ 3 กวาดบนคำถามดิบของผู้ใช้ (ตอบได้ 18 ข้อ, นอกขอบเขต 7 ข้อ) · คะแนนความคล้าย (cosine) อันดับ 1 ต่ำสุดของข้อที่ตอบได้คือ 0.54 ส่วนสูงสุดของข้อนอกขอบเขตคือ 0.55

ค่าตั้งต้น 0.40 ปฏิเสธข้อนอกขอบเขตได้เพียง 29% ส่วน 0.55 ปฏิเสธได้ทั้งหมดโดยยังตอบข้อที่ตอบได้ 94% (balanced accuracy 0.64 → 0.97) **แต่ช่องว่างบางมาก** (0.54 เทียบ 0.55) และกวาดจากคำถามของผู้ใช้ ไม่ใช่จากคำค้นที่เอเจนต์เขียนเอง จึงเป็นหลักฐานระดับ retrieval ไม่ใช่หลักฐานว่าระบบปลอดภัยกว่าในทุกกรณี

4.3 ทั้งระบบเทียบกับ baseline



รูปที่ 2 สัดส่วนคำถามที่ตอบได้ (29 ข้อ) ที่ตอบถูก (แท่งทึบ) และยืนยันด้วยแหล่งที่มาได้ (แท่งจาง)

การรัน	การตั้งค่า	ตอบถูก (จาก 29)	ยืนยันด้วยแหล่งที่มา	หลักฐานเอกสารที่ดึงได้ (12)	หลักฐานผสมที่ดึงได้ (6)	SQL ตรงเฉลย (11)	ปฏิเสธข้อนอกขอบเขต (7)	ปฏิเสธผิดข้อ
A ค่าตั้งต้นของแอปตัวอย่าง	dense, k=5, floor 0.40	19 (66%)	16 (55%)	5/12	3/6	9/11	7/7	0
B ปรับแล้ว (ค่าเริ่มต้น)	hybrid, k=10, floor 0.55	22 (76%)	20 (69%)	8/12	5/6	9/11	7/7	0
B2 รัน B ซ้ำ	เหมือน B	23 (79%)	21 (72%)	8/12	5/6	9/11	7/7	0
C prompt v2	B + กฎ routing	23 (79%)	22 (76%)	10/12	5/6	9/11	7/7	2
C2 รัน C ซ้ำ	เหมือน C	23 (79%)	22 (76%)	10/12	6/6	9/11	7/7	1
Baseline: เอกสารอย่างเดียว	B ไม่มีเครื่องมือ SQL	9 (31%)	8 (28%)	8/12	1/6	0/11	6/7	1
Baseline: โมเดลเปล่า	ไม่มีเครื่องมือเลย	5 (17%)	0 (0%)	0/12	0/6	0/11	0/7	0

ตารางที่ 4 ผลทั้งระบบ · "ปฏิเสธผิดข้อ" = ปฏิเสธคำถามที่ควรตอบ (จาก 29) · จุดหนึ่งของรัน C คือ proxy ตอบ rate limit (แก้ด้วยการ retry เพิ่มแล้ว)

การรัน	doc (12)	sql (11)	mixed (6)	oos (7)
Baseline: โมเดลเปล่า	5/12	0/11	0/6	0/7
Baseline: เอกสารอย่างเดียว	9/12	0/11	0/6	6/7
A ค่าตั้งต้นของแอปตัวอย่าง	7/12	10/11	2/6	7/7
B2 รัน B ซ้ำ	9/12	10/11	4/6	7/7

ตารางที่ 5 จำนวนข้อที่ตอบถูกแยกตามประเภทคำถาม

- **การปรับ A → B ได้ผลจริง:** ตอบถูก 19 → 22–23 ข้อ ยืนยันด้วยแหล่งที่มา 16 → 20–21 ข้อ หลักฐานเอกสารที่ดึงได้ 5/12 → 8/12 และข้อผสม 3/6 → 5/6 ตัวชี้วัดที่ไม่พึ่งตัวตัดสิน (หลักฐานที่ดึงได้, SQL ตรงเฉลย) ยืนยันทิศทางเดียวกัน
- **ความแกว่งของระบบ:** รัน B สองครั้งต่างกันเพียง 1 ข้อจาก 36 (`m02` ที่ตัวตัดสินพลิกบนคำตอบที่แทบเหมือนกัน) และตัวชี้วัดที่ไม่พึ่งตัวตัดสินเหมือนกันทุกตัว ความต่างระดับ 1–2 ข้อจึงอยู่ในระดับสัญญาณรบกวน

- **สิ่งที่ SQL เพิ่มให้ ไม่ใช่คำตอบเอกสาร:** ในข้อเอกสาร agent กับ "เอกสารอย่างเดียว" เสมอกัน (9/12) แต่ในข้อนับ/จัดอันดับและข้อผสม (17 ข้อ) **เอกสารอย่างเดียวถูก 0 ข้อ** เพราะ top-k นับไม่ได้ ส่วน agent ถูก 10/11 และ 3–4/6 (B และ B2) โมเดลเปล่าตอบเอกสารถูกจากความจำ 5/12 ข้อ แต่ไม่เคยอ้างอิงได้ ไม่เคยตอบข้อตารางได้ และ**ไม่ปฏิเสธเลย** (0/7)
- **เกณฑ์ floor ไม่เปลี่ยนอัตราปฏิเสธใน end-to-end:** ทั้ง A (0.40) และ B (0.55) ปฏิเสธข้อนอกขอบเขตได้ 7/7 เพราะโมเดลปฏิเสธเองอยู่แล้ว floor จึงเป็นตาข่ายนิรภัยระดับ retrieval ตามหัวข้อ 4.2 ไม่ใช่เหตุผลที่ 7 ข้อนี้ถูกปฏิเสธ
- **prompt v2 (เพิ่มกฎ routing 1 ข้อ) เป็นการแลกเปลี่ยน ไม่ใช่ชัยชนะ:** แก้ข้อ d03 ที่เลือกเครื่องมือผิดได้ และเพิ่มหลักฐานเอกสารที่ดึงได้จาก 8 เป็น 10/12 แต่ทั้งสองรอบ v2 ทำให้โมเดลปฏิเสธข้อ s06 (ร้อยละ) โดยอ้างว่า "เครื่องมือไม่รองรับการคำนวณร้อยละ" ตามเกณฑ์ที่ตั้งไว้ล่วงหน้า (ต้องไม่มีการปฏิเสธผิดข้อ) จึง**คง v1 เป็นค่าเริ่มต้น**

5. กรณีล้มเหลว (ตั้งค่า B)

F1 · s11 นับ CWE-78 ผิด (SQL ถูกกฎแต่ผิดความหมาย)
คำถามบอกชัดว่า "นับเฉพาะ CWE-78 ไม่รวมรหัสอื่นที่ขึ้นต้นด้วย 78" แต่โมเดลเขียน `SELECT COUNT(*) FROM kev WHERE cwes LIKE '%CWE-78%'` ได้ **220** (ตามที่เห็นในการรัน B2) ขณะที่ค่าที่ถูกคือ **112** เพราะ `LIKE` จับ CWE-787, CWE-789 ปนมาด้วย ข้อนี้พลาดในทุกการรัน (A, B, B2, C, C2)
ทำไมด่านป้องกันไม่จับได้: query นี้เป็น SELECT ที่อ่านตารางที่อนุญาต ผ่านด่านทุกชั้นอย่างถูกต้อง ด่านนี้รับประกัน*ความปลอดภัย* ไม่ใช่*ความถูกต้องเชิงความหมาย* ผู้ใช้เห็นแต่คำตอบที่ดูมั่นใจ จับได้ก็เพราะตัวประเมิน (execution match) เทียบกับ SQL เฉลย
แนวทางแก้: ทำ `cwes` เป็นตารางลูก `kev_cwe(cve_id, cwe)` ให้เปรียบเทียบเท่ากับแทนการค้นสตริง, ให้เอเจนต์สร้างคำสั่งตรวจสอบเช่นรัน query ที่สองด้วยถ้อยคำต่างกัน, หรือใช้โมเดลที่แรงขึ้นสำหรับขั้นเขียน SQL

F2 · d09 "NIST นิยามความเสี่ยงอย่างไร" ตอบมั่นใจแต่ผิด (grounded ไม่เท่ากับ correct)
ประโยคนิยามจริงอยู่อันดับเกิน 20 ในทุกวิธีค้น (BM25 อันดับ 960) เพราะคำในคำถามทั่วไปและปรากฏในทุกหน้าของ SP 800-30 เอเจนต์จึงได้ข้อความเกี่ยวกับ "กระบวนการประเมินความเสี่ยง" มาตอบเป็นนิยาม ซึ่งผิด แต่**มีแหล่งอ้างอิงรองรับ** ตัวชี้วัด "ยืนยันด้วยแหล่งที่มา" จับกรณีนี้ได้ (หลักฐานที่ถูกไม่ได้ถูกดึงมา) แต่ผู้ใช้จะไม่รู้ ข้อควรระวังคือ citation บอกว่าข้อความมาจากไหน ไม่ได้บอกว่าข้อความนั้นตอบคำถาม
แนวทางแก้: cross-encoder reranker, การเขียนคำค้นใหม่ก่อนค้น (L9), หรือใส่ชื่อเอกสารและหัวข้อนำหน้า chunk

F3 · d03 เลือกเครื่องมือผิด และ F4 · m05 ตัวตัดสินเข้มเกินไป
d03 ถามว่าคำสั่งของ CISA กำหนดกรอบเวลาแก้ช่องโหว่ไว้เท่าใด เอเจนต์เห็นคำว่า "known exploited vulnerabilities" แล้วไปรัน SQL ดึง `due_date` แถวแรก ได้คำตอบที่ไม่เกี่ยวกับคำถาม (v2 แก้ได้ แต่ทำให้ s06 พัง ดู 4.3) m05 ถูกนับว่าผิด แต่คำตอบระบุว่า image "จะไม่ทันสมัยอีกต่อไป" ซึ่งความหมายเท่ากับ "outdated" ในเฉลย: **ความผิดพลาดอยู่ที่ตัวตัดสิน ไม่ใช่โมเดลที่ตอบ**

ความน่าเชื่อถือของตัวตัดสิน ตัวตัดสินเป็นโมเดลเล็กตัวเดียวกับที่เขียนคำตอบ จึงมีโอกาสลำเอียงเข้าข้างตัวเอง ผลที่วัดได้: **นิ่ง** (35 จาก 35 คำตอบได้คำตัดสินเดิมเมื่อตัดสินซ้ำสามครั้ง) แต่ไวต่อถ้อยคำ ตรวจด้วยมือ 12 คำตอบของโมเดลเปล่า เห็นตรงกัน 10 ข้อ ที่เหลือสองข้อ: ข้อหนึ่งเฉลยอ้างอิงสั้นเกินไป (d12 แก้แล้วโดยเพิ่ม `judge_fact`) อีกข้อเป็นการตัดสิน "ไม่" ที่เข้มกว่าคนต่อคำที่แปลความเดียวกัน (d04 คงไว้) จึงคาดว่าคอลัมน์ "ตอบถูก" ฝั่งเอกสารต่ำกว่าความจริง 1–2 ข้อ ส่วนตัวชี้วัดที่ไม่พึ่งตัวตัดสิน (SQL ตรงเฉลย, หลักฐานที่ดึงได้, เลือกเครื่องมือ, การปฏิเสธ) ไม่ได้รับผลกระทบ

6. ข้อจำกัดและงานต่อไป

ข้อจำกัด

- **ตัวอย่างเล็ก และปรับค่าบนชุดเดียวกับที่รายงานผล:** chunking, วิธีค้น, `TOP_K` และ `min_score` ถูกเลือกจากคำถาม 36 ข้อชุดเดียวกับที่รายงาน มีเพียงการรัน A ที่ไม่ได้ปรับตามชุดนี้ ผลที่รายงานจึงอาจดีกว่าที่จะเห็นกับคำถามใหม่ และมีเพียง 18 ข้อที่วัด retrieval ได้ (1 ข้อ = 5.6 จุด)
- **ข้อนอกขอบเขตมีเพียง 7 ข้อ** และโมเดลปฏิเสธครบทั้งที่มีและไม่มี floor จึงยังไม่รู้ว่า floor ช่วยในกรณีที่โมเดลหลงตอบจริงหรือไม่

- **ภาษา:** ประเมินเป็นภาษาอังกฤษเท่านั้น ระบบถูกสั่งให้ตอบตามภาษาของคำถามและโมเดล embedding รองรับหลายภาษา แต่คุณภาพภาษาไทยยังไม่ได้วัด และตัวตัดคำของ BM25 ไม่รองรับอักษรที่ไม่ใช่ละติน
- **ข้อมูล:** KEV เป็น snapshot ที่เปลี่ยนทุกสัปดาห์ (เฉลยผูกกับ snapshot ที่ commit ไว้) PDF ของ NIST สกัดข้อความไม่สมบูรณ์ (ตารางแบน) และไม่มีประกาศของ CISA เพราะเว็บบล็อก client อัตโนมัติ
- **โมเดลเล็ก** เขียน SQL ที่ซับซ้อนหรือต้องตรงตัวผิดได้ (F1) และการรวม SQL กับการอ่านเอกสารในข้อเดียวยังพลาดบ่อย (ข้อผสมถูกเพียง 3–4 จาก 6)
- **การพึ่งพา proxy ร่วม:** สร้าง embedding ได้ราว 2.5 chunk/วินาที และประมวลผลทีละคำขอ (ส่งขนานไม่ช่วย) การสร้าง index หนึ่งชุดใช้ ~20 นาที จึงไม่ได้ทดสอบ `section:600` กับ dense ระบบที่ deploy ใช้แพ็กเกจฟรีที่หลับเมื่อไม่มีผู้ใช้ 15 นาที (เปิดครั้งแรกอาจรอ ~1 นาที) และตัวจำกัดอัตราอยู่ในหน่วยความจำของเครื่องเดียว

งานต่อไป (เรียงตามผลที่คาดว่าจะได้)

1. **ขยายชุดทดสอบและแยกชุดสำหรับรายงานผล** (เช่น 100+ ข้อ แบ่งชุดปรับค่า/ชุดทดสอบ) เป็นเงื่อนไขก่อนจะเชื่อตัวเลขใดๆ ในรายงานนี้ให้มากกว่าแนวโน้ม
2. **แก้ความผิดพลาดเชิงความหมายของ SQL** (F1): ทำให้ schema อยู่ในรูปที่ query ผิดได้ยาก, ตรวจคำตอบด้วย query ที่สอง, ลองโมเดลที่แรงขึ้นเฉพาะขั้นเขียน SQL
3. **ยกระดับ retrieval** (F2): cross-encoder reranker, query rewriting, หัวข้อนำหน้า chunk, ทดสอบ `section:600` กับ dense เมื่อ proxy เร็วขึ้น
4. **ประเมินภาษาไทย** พร้อมตัวตัดคำสำหรับ BM25 และชุดคำถามภาษาไทย
5. **เปิด `query_kev` เป็นเซิร์ฟเวอร์ MCP** ให้เอเจนต์ตัวอื่นใช้ร่วมกันได้ (L14) และเพิ่มแหล่งข้อมูล เช่น ประกาศเตือนภัยของหน่วยงานที่อนุญาตให้ดึงข้อมูลอัตโนมัติ

7. สรุป

โปรเจกต์นี้แสดงว่าเอเจนต์ขนาดเล็กที่มีสองเครื่องมือและด่านป้องกันที่ออกแบบดี ตอบคำถามที่ RAG อย่างเดียวตอบไม่ได้ (นับและจัดอันดับ) ได้ถูกต้อง และปฏิเสธนอกขอบเขตได้ดีในชุดทดสอบนี้ การปรับ retrieval (hybrid, k=10, floor 0.55) ให้ผลที่วัดได้ ส่วนการปรับ prompt และ chunking ไม่ให้ผลที่เชื่อถือได้ จุดอ่อนที่เห็นชัดไม่ใช่ความปลอดภัย แต่คือ**ความถูกต้องเชิงความหมายของ SQL**และ**การดึงนิยามกว้างๆ จากเอกสารยาว** ซึ่งทั้งสองกรณีโมเดลตอบด้วยความมั่นใจ จึงต้องมีกลไกตรวจสอบเพิ่มก่อนใช้งานจริง ตัวเลขทั้งหมดเป็นแนวโน้มจากตัวอย่างเล็ก ไม่ใช่การรับประกันประสิทธิภาพ

ภาคผนวก: ทำซ้ำผลการทดลอง

```
uv sync ; cp .env.example .env           # ใส่ OPENAI_API_KEY ใน .env
uv run python -m app.ingest              # สร้าง index (~20 นาที ใช้ key)
uv run python -m eval.run_eval check     # ตรวจชุดทดสอบ (ออฟไลน์)
uv run python -m eval.run_eval retrieval  # ตารางที่ 2
uv run python -m eval.run_eval floor     # ตารางที่ 3
uv run python -m eval.run_eval agent --modes agent --tag B_hybrid_k10_floor055   # ตารางที่ 4 (ตั้งค่าตาม .env)
uv run pytest -q                         # เทสต์ 89 ตัว ไม่ใช้ key
```

ผลดิบทุกการรัน (คำตอบ, SQL ที่รัน, คำตัดสินรายข้อ) อยู่ที่ `eval/results/*.json` · แหล่งข้อมูล: CISA Known Exploited Vulnerabilities Catalog (CC0 1.0) และเอกสาร NIST 12 เล่ม (ผลงานรัฐบาลสหรัฐ) รายการและที่มาครบถ้วนใน `data/SOURCES.md`